

Sistem Manajemen Tiket Bioskop (CinemaTicketCLI)

1. Latar Belakang

Industri perbioskopian membutuhkan sistem pengelolaan data film dan pemesanan tiket yang cepat, akurat, dan konsisten. Proses pencatatan secara manual rentan terhadap kesalahan, seperti duplikasi kursi atau data pemesanan yang tidak sinkron dengan ketersediaan kursi aktual. Oleh karena itu, dibutuhkan sebuah aplikasi yang dapat mengelola data film, transaksi pemesanan, dan pembatalan tiket secara terintegrasi dengan basis data.

Project ini dikembangkan sebagai tugas mata kuliah Object-Oriented Programming (OOP), dengan tujuan menerapkan konsep pemrograman berorientasi objek sekaligus mengintegrasikannya dengan fitur-fitur basis data tingkat lanjut, seperti stored procedure, function, trigger, dan view, pada studi kasus nyata berupa sistem tiket bioskop.

2. Tujuan dan Fungsi Aplikasi

Aplikasi CinemaTicketCLI bertujuan untuk menyediakan sarana pengelolaan data film dan transaksi tiket bioskop melalui antarmuka Command Line Interface (CLI), dengan seluruh data tersimpan secara persisten pada basis data MySQL/MariaDB menggunakan JDBC. Secara garis besar, aplikasi ini memiliki fungsi-fungsi berikut:

- Manajemen film — menambahkan data film baru dan menampilkan daftar film yang tersedia.
- Pemesanan tiket — melakukan pemesanan tiket untuk suatu film dengan validasi ketersediaan kursi.
- Pembatalan tiket — membatalkan tiket yang telah dipesan, dengan kursi yang dibebaskan kembali secara otomatis.
- Pelaporan — menampilkan riwayat seluruh pemesanan serta daftar tiket yang masih berstatus aktif.

Dengan fungsi-fungsi tersebut, aplikasi diharapkan dapat menyederhanakan alur kerja pengelolaan tiket bioskop sekaligus menjamin konsistensi data ketersediaan kursi melalui mekanisme basis data otomatis (trigger).

3. Implementasi Konsep Object-Oriented Programming (OOP)

Aplikasi ini dibangun dengan menerapkan seluruh pilar utama OOP, yang diwujudkan pada struktur package dan class sebagai berikut:

3.1 Class dan Object

Sistem terdiri atas beberapa class utama, yaitu Movie, PremiumMovie, Ticket, dan Customer sebagai entity, MovieDAO dan TicketDAO sebagai lapisan akses data, DatabaseConnection untuk koneksi basis data, serta CinemaService dan Main sebagai lapisan logika bisnis dan antarmuka pengguna. Object dari class-class tersebut dibuat dan digunakan secara dinamis selama aplikasi berjalan, misalnya setiap kali pengguna menambahkan film atau melakukan pemesanan tiket.

3.2 Inheritance (Pewarisan)

Class Movie berperan sebagai superclass yang menyimpan atribut dan perilaku umum sebuah film. Class PremiumMovie diturunkan (extends) dari Movie dan mewarisi seluruh atribut serta method induknya, sekaligus menambahkan atribut khusus berupa biaya tambahan (extra price) untuk kategori film premium.

3.3 Polymorphism

Polymorphism diterapkan melalui method displayInfo() pada class Movie, yang di-override oleh class PremiumMovie. Ketika dipanggil pada object PremiumMovie, method ini tidak hanya menampilkan informasi dasar film, tetapi juga informasi tambahan berupa kategori dan biaya tambahan, tanpa mengubah cara pemanggilan method dari sisi pemanggil.

3.4 Encapsulation

Seluruh atribut pada class entity (Movie, Ticket, Customer, dan sejenisnya) dideklarasikan sebagai private, sehingga tidak dapat diakses langsung dari luar class. Akses terhadap atribut tersebut hanya dapat dilakukan melalui method getter dan setter, sehingga integritas data lebih terjaga dan validasi dapat diterapkan secara terpusat.

3.5 Package

Struktur kode diorganisasikan ke dalam beberapa package berdasarkan tanggung jawabnya masing-masing, yaitu entity (representasi data), dao (akses basis data), database (koneksi basis data), service (logika bisnis), util (fungsi bantu input), dan main (titik masuk aplikasi). Pemisahan ini menerapkan prinsip separation of concerns sehingga kode lebih terstruktur dan mudah dikelola.

3.6 Exception Handling

Aplikasi menangani berbagai kemungkinan kesalahan menggunakan mekanisme try-catch, di antaranya SQLException pada seluruh proses akses basis data, InputMismatchException dan NumberFormatException pada proses input pengguna, serta validasi tambahan untuk kondisi kursi penuh

dan data input yang kosong. Penanganan ini memastikan aplikasi tetap berjalan stabil dan memberikan pesan yang informatif kepada pengguna ketika terjadi kesalahan.

3.7 Integrasi dengan Basis Data Lanjutan

Selain konsep OOP pada sisi Java, aplikasi juga memanfaatkan fitur basis data lanjutan pada MySQL/MariaDB, yaitu stored procedure `add_movie` untuk menambahkan data film, function `remaining_seats` untuk menghitung sisa kursi, trigger `reduce_seat_after_booking` dan `increase_seat_after_cancellation` untuk memperbarui jumlah kursi secara otomatis, serta view `active_ticket_view` untuk menampilkan ringkasan tiket aktif. Integrasi ini dilakukan melalui JDBC pada lapisan DAO, menggunakan `PreparedStatement` untuk query pada umumnya dan `CallableStatement` untuk memanggil stored procedure.

4. Tampilan Aplikasi

Berikut tampilan menu utama serta setiap submenu aplikasi CinemaTicketCLI beserta penjelasan fungsinya. Screenshot diambil langsung dari terminal saat aplikasi dijalankan.

4.1 Menu Utama

Menu utama ditampilkan setiap kali aplikasi dijalankan atau setelah suatu proses selesai. Menu ini menjadi pusat navigasi ke seluruh fitur aplikasi (menu 1 sampai 6, serta opsi keluar).

```
===== CINEMA TICKET CLI =====  
1. Tambah Film  
2. Lihat Daftar Film  
3. Pesan Tiket  
4. Batalkan Tiket  
5. Lihat Riwayat Pemesanan  
6. Lihat Tiket Aktif  
0. Keluar  
Pilih menu: |
```

4.2 Menu 1 — Tambah Film

Digunakan untuk menambahkan data film baru ke basis data. Pengguna memasukkan judul, genre, durasi, dan jumlah kursi, yang kemudian disimpan melalui pemanggilan stored procedure `add_movie`.

```
Pilih menu: 1
Judul film: Project Hail Mary
Genre: Astronomi, Adventure, Sci-fi
Durasi (menit): 182
Jumlah kursi: 100
Film berhasil ditambahkan.
```

4.3 Menu 2 — Lihat Daftar Film

Menampilkan seluruh data film yang tersimpan di basis data, lengkap dengan informasi genre, durasi, dan sisa kursi tersedia.

```
Pilih menu: 2
ID: 1
Judul: Avengers: Doomsday
Genre: Action
Durasi: 150 menit
Kursi tersedia: 99
-----
ID: 2
Judul: Project Hail Mary
Genre: Astronomi, Adventure, Sci-fi
Durasi: 182 menit
Kursi tersedia: 100
-----
```

4.4 Menu 3 — Pesan Tiket

Digunakan untuk memesan tiket suatu film. Sistem memvalidasi ketersediaan kursi menggunakan function `remaining_seats` sebelum data pelanggan dan tiket disimpan; jumlah kursi tersedia otomatis berkurang melalui trigger `reduce_seat_after_booking`.

```
===== CINEMA TICKET CLI =====
1. Tambah Film
2. Lihat Daftar Film
3. Pesan Tiket
4. Batalkan Tiket
5. Lihat Riwayat Pemesanan
6. Lihat Tiket Aktif
0. Keluar
Pilih menu: 3
ID Film: 2
Nama pelanggan: djibriel
No. HP: 08123123
Nomor kursi: A2
Tiket berhasil dipesan untuk djibriel di kursi A2
```

4.5 Menu 4 — Batalkan Tiket

Digunakan untuk membatalkan tiket yang sudah dipesan berdasarkan ID tiket. Status tiket diubah menjadi CANCELLED, dan kursi yang tadinya terpakai otomatis dikembalikan melalui trigger `increase_seat_after_cancellation`.

```
===== CINEMA TICKET CLI =====
1. Tambah Film
2. Lihat Daftar Film
3. Pesan Tiket
4. Batalkan Tiket
5. Lihat Riwayat Pemesanan
6. Lihat Tiket Aktif
0. Keluar
Pilih menu: 4
ID Tiket yang dibatalkan: 3
Tiket berhasil dibatalkan.
```

4.6 Menu 5 — Lihat Riwayat Pemesanan

Menampilkan seluruh riwayat tiket yang pernah dibuat, baik yang masih aktif maupun yang sudah dibatalkan, sebagai bentuk laporan transaksi secara keseluruhan.

```
===== CINEMA TICKET CLI =====
1. Tambah Film
2. Lihat Daftar Film
3. Pesan Tiket
4. Batalkan Tiket
5. Lihat Riwayat Pemesanan
6. Lihat Tiket Aktif
0. Keluar
Pilih menu: 5
ID:1 | Movie:1 | Customer:0 | Kursi:A1 | Status:CANCELLED
ID:2 | Movie:1 | Customer:1 | Kursi:B5 | Status:ACTIVE
ID:3 | Movie:2 | Customer:2 | Kursi:A2 | Status:ACTIVE
```

4.7 Menu 6 — Lihat Tiket Aktif

Menampilkan daftar tiket yang berstatus aktif saja, memanfaatkan view `active_ticket_view` yang sudah menggabungkan data tiket, film, dan pelanggan dalam satu tampilan.

```
===== CINEMA TICKET CLI =====
1. Tambah Film
2. Lihat Daftar Film
3. Pesan Tiket
4. Batalkan Tiket
5. Lihat Riwayat Pemesanan
6. Lihat Tiket Aktif
0. Keluar
Pilih menu: 6
Ticket ID : 2
Film      : Avengers: Doomsday
Pelanggan : Budi Santoso
Kursi     : B5
Tanggal   : 2026-07-19
-----
Ticket ID : 3
Film      : Project Hail Mary
Pelanggan : djibriel
Kursi     : A2
Tanggal   : 2026-07-19
-----
```

5. Ringkasan Struktur Package

Package	Isi / Tanggung Jawab
entity	Movie, PremiumMovie, Ticket, Customer — representasi data
dao	MovieDAO, TicketDAO — akses dan manipulasi data pada basis data
database	DatabaseConnection — pengelolaan koneksi JDBC
service	CinemaService — logika bisnis aplikasi
util	InputHelper — validasi dan pembacaan input pengguna
main	Main — menu CLI dan titik masuk aplikasi

6. Kesimpulan

6.1 Hasil Implementasi

Pengembangan aplikasi CinemaTicketCLI berhasil menghasilkan sistem CLI yang dapat mengelola data film, memproses pemesanan dan pembatalan tiket, serta menampilkan laporan riwayat dan tiket aktif. Seluruh konsep OOP yang ditargetkan — class, object, inheritance, polymorphism, encapsulation, package, dan exception handling — berhasil diterapkan pada struktur kode. Di sisi basis data, seluruh fitur dasar (CREATE, INSERT, SELECT, UPDATE, DELETE) maupun lanjutan (stored procedure add_movie, function remaining_seats, trigger reduce_seat_after_booking dan increase_seat_after_cancellation, serta view active_ticket_view) telah diuji dan berfungsi sebagaimana mestinya, dengan jumlah kursi tersedia yang selalu konsisten secara otomatis setiap kali terjadi pemesanan maupun pembatalan.

6.2 Kendala yang Dihadapi dan Solusi

Selama proses pengembangan, ditemukan beberapa kendala teknis sebagai berikut, beserta solusi yang diterapkan untuk mengatasinya:

- Kendala: Penggunaan DELIMITER saat membuat procedure, function, dan trigger di MariaDB CLI sering menyebabkan kesalahan sintaks maupun sesi yang "menggantung" (prompt tidak kembali normal).

- Solusi: Memahami bahwa DELIMITER yang diubah (mis. menjadi \$\$) harus dikembalikan ke ; setelah blok BEGIN...END selesai dibuat, serta memastikan penulisan DELIMITER dilakukan pada baris terpisah tanpa tanda titik koma di belakangnya.
- Kendala: Perintah kompilasi menggunakan wildcard (contoh: src/entity/*.java) tidak dikenali oleh PowerShell, sehingga proses javac gagal.
 - Solusi: Mengganti pendekatan dengan membuat daftar seluruh berkas .java menggunakan Get-ChildItem -Recurse, kemudian mengompilasi berkas tersebut melalui parameter @sources.txt.
- Kendala: Berkas daftar sumber (sources.txt) yang dihasilkan PowerShell menggunakan encoding yang tidak kompatibel dengan javac, sehingga muncul kesalahan MalformedInputException saat kompilasi.
 - Solusi: Menyimpan berkas daftar sumber dengan encoding UTF-8 secara eksplisit menggunakan parameter -Encoding utf8 pada perintah Out-File.
- Kendala: Struktur package awal sempat menyimpang dari rancangan awal karena penambahan class DAO baru (CustomerDAO) yang tidak sesuai spesifikasi.
 - Solusi: Menyesuaikan kembali implementasi dengan memindahkan logika penambahan data pelanggan ke dalam TicketDAO, sehingga struktur package tetap konsisten dengan rancangan yang telah ditetapkan di awal.

6.3 Penutup

Secara keseluruhan, kendala yang muncul selama pengembangan lebih banyak bersifat teknis pada tahap konfigurasi lingkungan pengembangan (basis data dan proses kompilasi), bukan pada logika maupun rancangan aplikasi itu sendiri. Dengan solusi yang diterapkan, aplikasi CinemaTicketCLI dapat berjalan sesuai dengan tujuan dan fungsi yang telah dirancang, sekaligus menjadi sarana penerapan konsep OOP secara terintegrasi dengan basis data relasional.